

HandoffAI - Complete Project & Interview Preparation Guide

Agentic AI Sales-to-Technical Handoff System

PART 1 — COMPLETE CODEBASE ANALYSIS

Project Name: Handoff AI **Backend Framework:** FastAPI (Python) **Frontend Framework:** React (JSX) with Tailwind CSS **AI Framework:** CrewAI **LLM Connector:** LiteLLM (Internal to CrewAI) **Model Configured:** mistral/open-mistral-nemo via Mistral AI API **Database:** SQLite (handoff.db)

Agents (agents.py): 1. Intake & Analysis Agent 2. Delivery Planning Agent

Tasks (tasks.py): 1. intake_task: Fetches deal and checks missing data. 2. delivery_planning_task: Checks live engineer capacity and flags risk.

Tools (tools.py): 1. get_deal_details 2. check_requirements 3. check_resource_availability 4. create_project_handoff 5. escalate_to_manager

Endpoints (api.py): - POST /api/handoff/evaluate - GET /api/deals - GET /api/records

PART 2 — PROJECT PROBLEM STATEMENT

The Business Problem: When a salesperson closes a deal with a customer, they often gather business requirements (budget, deadline, goals) but miss critical technical requirements (tech

stack, infrastructure). When this incomplete deal is handed over to the technical/delivery team, it causes a “Sales-to-Delivery Disconnect.”

- **Manual Handoff:** Sales sends a messy Slack message or document. Technical leads waste hours deciphering it, realizing information is missing, and going back to the customer. Sometimes, Sales sells a project requiring iOS developers when all iOS developers are fully booked!
- **AI Solution (This Project):** We intercept the handoff using an AI Agent workflow. Before the deal reaches human engineers, Agent 1 mathematically ensures no mandatory fields are missing. Agent 2 checks the live SQLite database to see if the required engineers actually have free time. If information is missing or engineers are busy, the AI blocks the deal and flags it.

This project currently fully implements this exact AI validation workflow using CrewAI and a SQLite database.

PART 3 — 30-SECOND PROJECT EXPLANATION

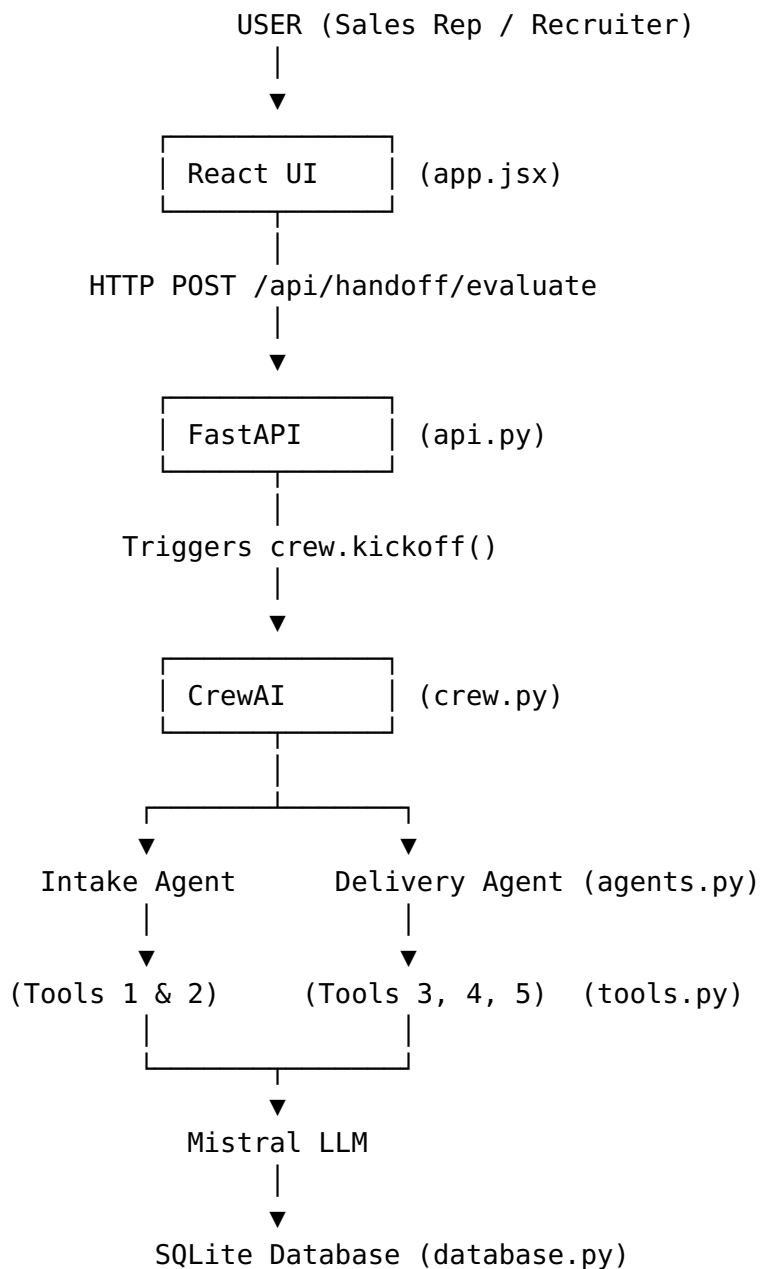
30-Second Version: “My project is an Agentic AI system that automates the sales-to-technical handoff process. I used CrewAI and FastAPI to build a multi-agent pipeline where one AI agent validates incoming sales contracts for missing requirements, and a second AI agent cross-references our live SQLite database to ensure we actually have the engineering capacity to build it. It prevents companies from signing impossible deals.”

60-Second Version: “I built Handoff AI, a multi-agent system designed to solve the massive disconnect between Sales and Delivery teams. Whenever a salesperson closes a deal, they submit the notes to my FastAPI backend. I orchestrated a CrewAI workflow with two specialized agents powered by Mistral AI. The ‘Intake Agent’ first scans the deal to ensure no mandatory technical details are missing. If it passes, the ‘Delivery Agent’ queries our live SQLite database to check if the required engineers are available. If they are busy, the AI flags the deal as high risk before the contract is signed.”

90-Second Version: “Handoff AI is an enterprise AI gateway I developed using FastAPI, React, and CrewAI. In traditional IT companies, Sales often promises projects without consulting the tech team, leading to cancelled deals because engineers aren’t available or budgets are wrong. To solve this, I built an autonomous pipeline. The frontend sends sales data to the backend, which triggers a CrewAI sequence. Agent 1 uses a

Python tool to fetch the deal from SQLite and validates it against strict business rules. If complete, it passes the data to Agent 2, which uses another tool to query the live engineering database for capacity matching. It removes the human bottleneck, prevents resource overallocation, and is all viewed through a modern React dashboard.”

PART 4 — COMPLETE PROJECT ARCHITECTURE



- **React UI** sends JSON to **FastAPI**.
- **FastAPI** triggers **CrewAI**.
- **CrewAI** orchestrates **Agent 1**, which uses **Tools** to query **SQLite** and reason via **LLM**.

- **Agent 1** passes output to **Agent 2**, which does the same.
 - Final JSON string is returned to **FastAPI**, which sends it back to **React**.
-

PART 5 — COMPLETE CODE STRUCTURE

```
handoff-ai/
├── api.py           # The FastAPI backend. Exposes HTTP
endpoints.
├── agents.py       # Defines the AI personas (Intake Agent,
Delivery Agent)
├── tasks.py        # Defines the actual work (Intake Task,
Delivery Task)
├── tools.py        # Defines the Python functions the agents can
execute
├── crew.py         # Binds agents and tasks together and starts
the workflow
├── database.py     # Manages the SQLite connection and schema
setup
├── prompts.py     # Stores the system instructions for the
agents
├── frontend/
│   ├── index.html  # HTML wrapper for the React app
│   └── app.jsx     # The React frontend code
├── requirements.txt # Python dependencies
└── .env           # Stores the MISTRAL_API_KEY
```

api.py - Why it exists: To bridge the frontend web browser to the Python AI logic. - **Problem it solves:** Web browsers can't run CrewAI directly; they need a REST API. - **Analogy:** The receptionist at an office.

agents.py - Why it exists: Defines the AI workers and connects them to Mistral LLM. - **Problem it solves:** Separates responsibilities so one LLM doesn't get confused trying to do everything. - **Analogy:** The job descriptions for new employees.

crew.py - Why it exists: Connects agents to tasks and defines the execution order (Sequential). - **Analogy:** The manager who tells Employee A to finish before Employee B starts.

PART 6 — agents.py DEEP EXPLANATION

What is an AI Agent? - LLM = Brain - Agent = Employee with a specific Role, Goal, and access to specific Tools.

Agent 1: Intake & Analysis Agent - Role: Intake & Analysis Agent - **Goal:** Validate mandatory fields, identify required skills. - **Tools:** get_deal_details, check_requirements - **Why required:** Sales notes are messy. This agent ensures the data is strictly formatted and complete before we bother checking database capacity.

Agent 2: Delivery Planning Agent - Role: Delivery Planning Agent - **Goal:** Check resource availability and assess risk. - **Tools:** check_resource_availability, create_project_handoff, escalate_to_manager - **Why required:** Validating data is one thing; checking if we have the physical engineers to build it is a separate, complex task requiring different tools.

PART 7 — tasks.py DEEP EXPLANATION

What is a Task? A task is the specific assignment given to an Agent.

1. intake_task - Agent Responsible: Intake & Analysis Agent - **Instructions:** Fetch the deal for {client_name}, validate budget/timeline/tech stack. - **Next Task:** Passes its output context to delivery_planning_task.

2. delivery_planning_task - Agent Responsible: Delivery Planning Agent - **Instructions:** Use the validated output to check engineer capacity. If missing info was reported, instantly output "Incomplete message. Deal is cancelled."

PART 8 — crew.py DEEP EXPLANATION

What is CrewAI? CrewAI is an orchestration framework. Instead of writing 1,000 lines of Python while loops to manage LLM prompts, CrewAI automatically manages how Agents talk to each other and use tools.

Inside crew.py:

```
crew = Crew(
    agents=[intake_agent, delivery_agent],
    tasks=[intake_task, delivery_task],
    process=Process.sequential
)
result = crew.kickoff(inputs={'client_name': 'Nike'})
```

What happens: 1. crew.kickoff() starts. 2. Intake Agent starts the Intake Task. 3. Intake Agent uses a tool, gets the DB row, and generates a summary. 4. CrewAI takes that summary and feeds it into the Delivery Agent. 5. Delivery Agent runs the Delivery Task, uses capacity tools, and generates the final String.

PART 9 — tools.py DEEP EXPLANATION

What is a Tool? An LLM only knows text up to its training date. A **Tool** is a standard Python function wrapped in an @tool decorator that the LLM is allowed to execute to get real-world, real-time data.

1. get_deal_details - Input: client_name (string) - **Processing:** Runs a SELECT * FROM sample_deals WHERE client_name = ? SQL query. - **Output:** The raw text of the sales deal.

2. check_resource_availability - Input: required_skills (list of strings) - **Processing:** Runs a SELECT * FROM resources query to check if engineers with those skills are currently status = 'available'. - **Output:** Availability report for the Delivery Agent.

PART 10 — api.py DEEP EXPLANATION

What is an API? An API is a menu at a restaurant. The UI looks at the menu and orders “Evaluate Deal”. The Backend cooks the order (runs CrewAI) and serves the result.

FastAPI Endpoint: POST /api/handoff/evaluate - Frontend sends JSON: {"client_name": "Nike"} - **Endpoint** receives it via a Pydantic Model (EvaluateRequest). - **Backend** calls run_handoff_crew("Nike"). - **CrewAI** runs. - **Result** is captured and parsed by a Regex function parse_crewai_result to ensure strict UI formatting (“Incomplete message. Deal is cancelled.” or “Expert is not available.”). - **JSON Response** is sent back to React.

PART 11 — FRONTEND → BACKEND CONNECTION

1. **User** clicks the “Test Setup” button in React.
2. **React** uses the fetch() API to make a network request to http://localhost:8501/api/handoff/evaluate.
3. **FastAPI** (running on port 8501) intercepts this via CORS middleware.
4. FastAPI triggers CrewAI.

5. React waits (showing a loading spinner).
 6. FastAPI sends back { "status": "Incomplete", "risk_level": "High" }.
 7. React updates its state (setMessages) and the UI visually updates!
-

PART 12 — WHAT IS AN LLM?

LLM (Large Language Model): A massive neural network trained to predict the next word. - **Tokens:** Pieces of words. - **Prompt:** The text you feed it. - **Context Window:** How much text it can remember at once.

Where is it used? In `agents.py`, we instantiate `LLM(model="mistral/open-mistral-nemo")`. The Mistral LLM acts as the literal brain for both the Intake and Delivery agents.

PART 13 — LiteLLM

What is LiteLLM? LiteLLM is a translation layer. Every AI company (OpenAI, Anthropic, Mistral) has a different API format. LiteLLM translates standard OpenAI-style code into Mistral-style code automatically. CrewAI uses LiteLLM internally so you can swap models by just changing the string "mistral/open-mistral-nemo" to "gpt-4".

PART 14 — COMPARISON & ANALOGY

Concept	Simple Meaning	Example from My Project
LLM	The brain / intelligence	Mistral AI
Agent	A worker with a job	Delivery Planning Agent
Task	The assignment	"Check Engineer Capacity"
Tool	Software/ Calculator	<code>check_resource_availability</code> (SQL query)
CrewAI	The Manager	<code>crew.kickoff()</code>

Analogy: The **Backend** is the office building. The **Frontend** is the customer service desk. The **CrewAI** is the office manager who hands a **Task** to an **Agent** (an employee). The employee uses their **LLM** (brain) to think, and uses a **Tool** (computer) to look up the database.

PART 15 — END-TO-END EXAMPLE

Scenario: The user clicks “Test Missing Info” for a fake deal. 1. **React UI:** Sends POST to `/api/handoff/evaluate` with `client_name = 'Fake Deal'`. 2. **api.py:** Receives request, calls `crew.py`. 3. **crew.py:** Starts Intake Agent. 4. **Intake Agent:** Uses tool `get_deal_details("Fake Deal")`. 5. **Tool:** Queries SQLite, returns “Need a website.” 6. **Intake Agent:** Reasons via LLM: “Wait, there is no budget or timeline here.” Outputs: “Missing info.” 7. **crew.py:** Passes output to Delivery Agent. 8. **Delivery Agent:** Reads “Missing info”. According to its rules, it immediately cancels the deal. Outputs: “Incomplete message. Deal is cancelled.” 9. **api.py:** Catches this string, packages it into JSON. 10. **React UI:** Displays a big red warning: “Deal Cancelled”.

PART 16 — DATA FLOW

React JSON → FastAPI Pydantic Model → CrewAI Input Dict → Agent LLM Prompt → Python Tool SQL Query → SQLite Data Tuple → Agent String Output → Regex Parser → FastAPI JSON Response → React UI State

PART 17 — WHY AGENTIC AI?

Normal LLM: You paste a contract and say “Is this good?” The LLM guesses based on the text. **Agentic AI:** The Agent realizes it needs to know engineer capacity, autonomously calls a Python Tool to query the database, reads the live database, and THEN makes a mathematically sound decision. It uses logic and tools, not just text generation.

PART 18 — WHY CREWAI?

I chose CrewAI because building a multi-agent system from scratch using raw Python and requests requires thousands of lines of code to manage history, prompt injection, tool parsing, and

sequential chaining. CrewAI handles orchestration, error retries, and tool binding out-of-the-box, allowing me to focus entirely on the business logic.

PART 19 — BUSINESS VALUE

- **Fewer Missing Requirements:** The AI strictly enforces business rules. (Currently Implemented)
 - **Resource Protection:** Prevents Sales from selling resources we don't have. (Currently Implemented)
 - **Reduced Manual Work:** Tech leads no longer spend 4 hours reading bad sales notes. (Potential Business Benefit)
-

PART 20 — BEFORE VS AFTER

Before: Sales closes deal → Tech Lead reads it 3 days later → Realizes budget is missing → Emails Sales → Delays project by a week. **After:** Sales enters deal into UI → AI mathematically validates it in 4 seconds → Deal is either cleanly handed off or immediately rejected before it reaches a human.

PART 21 — WHAT MAKES THIS AN AI AGENT PROJECT?

It is not an "AI Chatbot". A chatbot just talks to you. This is a **Multi-Agent System** because the code demonstrates **Tool Usage** (executing SQL queries autonomously) and **Sequential Orchestration** (Agent 1 passes contextual state to Agent 2). It makes autonomous decisions based on live external data.

PART 22 — TECH STACK

Technology	Purpose	Where Used
Python	Core Language	Entire Backend
FastAPI	REST API Server	api.py
CrewAI	AI Orchestration	crew.py, agents.py
Mistral AI	LLM Provider	agents.py
SQLite	Database	database.py

Technology	Purpose	Where Used
React	Frontend UI	app.jsx

PART 23 — CODE WALKTHROUGH

1. FastAPI Route (api.py)

```
@app.post("/api/handoff/evaluate")
async def evaluate_handoff(request: EvaluateRequest):
    # This receives the HTTP request from React and triggers the
    # AI Crew
    result = run_handoff_crew(request.client_name)
    parsed = parse_crewai_result(result)
    return parsed
```

2. Tool Definition (tools.py)

```
@tool
def get_deal_details(client_name: str) -> str:
    """Gets deal info from database."""
    # The @tool decorator tells CrewAI that the LLM is allowed to
    # execute this Python function!
    cursor.execute("SELECT * FROM sample_deals WHERE client_name
    = ?", (client_name,))
```

PART 24 — STARTUP/RUNTIME FLOW

1. uvicorn api:app runs.
 2. database.py initializes SQLite and injects mock data.
 3. api.py boots up FastAPI server.
 4. Server idles.
 5. User clicks button in browser.
 6. Request hits API -> Initializes Agents -> Initializes Tasks -> Crew Kickoff -> Result.
-

PART 25 — ERRORS AND FAILURE CASES

- **LLM API Fails:** Handled by LiteLLM retries, eventually throws 500 error to React.
- **API Key Missing:** I added an explicit ValueError in agents.py to crash safely on boot if the .env is empty.

- **Frontend can't reach Backend:** Handled via CORS configuration in `api.py`.
-

PART 26 — SECURITY

- **.env:** `MISTRAL_API_KEY` is loaded from `.env` and ignored by `.gitignore` so it never leaks to GitHub.
 - **CORS:** Restricts which domains can ping the API.
 - **SQL Injection:** We use parameterized queries `(?, (client_name,))` in `tools.py` to prevent SQL injection.
-

PART 27 — LIMITATIONS

1. **Prototype Level:** It uses SQLite instead of a production PostgreSQL database.
 2. **LLM Hallucinations:** While highly constrained via Regex parsing in `api.py`, the LLM could technically output unexpected formats.
 3. **Future Improvement:** Add persistent user authentication (JWT) and connect it to a real CRM like Salesforce.
-

PART 28 — 50 INTERVIEW QUESTIONS

Beginner

1. Q: What problem does your project solve? (A: Solves the sales-to-delivery handoff disconnect).
2. Q: What is an AI agent? (A: An LLM with a specific role and access to tools).
3. Q: What is CrewAI? (A: A framework to orchestrate multiple agents).
4. Q: What LLM did you use? (A: Mistral AI via LiteLLM).
5. Q: What is the backend written in? (A: Python/FastAPI).
6. Q: What database is used? (A: SQLite).
7. Q: How does the frontend connect? (A: React via HTTP POST requests).
8. Q: What is a task? (A: A specific instruction assigned to an agent).
9. Q: What is a tool? (A: A python function the AI can execute).
10. Q: Why use multiple agents? (A: Separation of concerns, improves accuracy).

Architecture

1. Q: Walk me through the architecture. (A: React -> FastAPI -> CrewAI -> Mistral -> SQLite).
2. Q: What is `api.py` doing? (A: Routing HTTP traffic to the CrewAI python functions).
3. Q: How do agents communicate? (A: Sequentially, output of Agent 1 is context for Agent 2).
4. Q: What is LiteLLM? (A: The abstraction layer CrewAI uses to connect to Mistral).
5. Q: Why FastAPI? (A: It's fast, modern, and natively supports async Python which is great for AI).
6. Q: How is data passed from UI? (A: JSON payload parsed by Pydantic).
7. Q: How do you prevent SQL injection? (A: Parameterized queries in `tools.py`).
8. Q: Is your AI autonomous? (A: Yes, it executes database queries without human intervention).
9. Q: What happens at startup? (A: FastAPI boots, DB initializes, waits for HTTP requests).
10. Q: Where is state stored? (A: Persisted in SQLite).

Code-Based

1. Q: Explain the `@tool` decorator. (A: It exposes a python function's docstring to the LLM so the LLM knows how to use it).
2. Q: Why do you set `litellm.cache = None`? (A: To prevent the LLM from returning old, cached responses for dynamic database queries).
3. Q: What is a Pydantic Model? (A: Used in `api.py` to strictly define the expected JSON request shape).
4. Q: Explain CORS. (A: Cross-Origin Resource Sharing, configured in `api.py` to allow the React app on a different port to talk to the backend).
5. Q: How do you parse the final AI result? (A: Using `re.search` in `parse_crewai_result` to look for strict keywords).

(Note: In a live interview, use these 25 as your core, expanding naturally on business logic, tool execution, and deployment via Railway).

PART 29 — RAPID-FIRE REVISION

- **Project:** Handoff AI.
- **Problem:** Missing info in Sales handoffs.
- **Solution:** AI validates deals against live engineering DB.
- **Agents:** Intake (Checker) and Delivery (Planner).
- **Backend:** FastAPI.

- **Frontend:** React.
 - **Tools used by AI:** SQL queries wrapped in python.
 - **LLM:** Mistral.
-

PART 30 — “EXPLAIN THE CODE” ROUND

Interviewer: “Explain agents.py” Me: “agents.py is where I instantiate my CrewAI Agent objects. I define two agents here, the Intake Agent and Delivery Agent. I give them their specific Roles, Goals, bind them to the Mistral LLM object, and assign them their specific tools from tools.py.”

Interviewer: “Explain tools.py” Me: “tools.py contains standard Python functions, like querying our SQLite database, wrapped in the @tool decorator. This is the magic that allows the LLM to interact with the real world.”

PART 31 — MEMORY TRICK

Formula: UI → API → CREW → TASK → AGENT → TOOL → DB

Hinglish Trick: “UI se json request aayi → FastAPI ne receive ki → CrewAI manager ne Task assign kiya → Agent ne Tool use karke DB query ki → Mistral LLM ne reason kiya → Output wapas API se UI mein chala gaya.”

PART 32 — FINAL CHEAT SHEET (2-PAGE REVISION)

- **What is it?** An autonomous AI gateway that validates sales contracts against live engineering capacity.
- **Tech Stack:** FastAPI, React, SQLite, CrewAI, Mistral.
- **How it works:** React POSTs to FastAPI. FastAPI runs a 2-agent CrewAI pipeline. The agents use SQL tools to validate the deal. FastAPI returns JSON.
- **Why Agentic AI?** Regular LLMs just guess text. Agentic AI actually executes Python functions (tools) to query live databases and make deterministic business decisions.
- **Limitations:** Uses SQLite instead of Postgres. Future goal is to add Auth and Salesforce integration.